WildCard 타일 시스템 네트워크 최적화 및 보안 아키텍처 보고서

종합 리팩토링 보고서: 11개 대상 파일 전체 수정 내역 및 핵심 코드 기술 명세

프로젝트: WildCard (UE 5.7 Dedicated Server) **문서 분류:** Core Engine & Network Architecture **일자:** 2026-10-03 **빌드 검증:** WildCardEditor (Success)

1. 변경된 파일 목록 및 핵심 작업 요약

타일 시스템 리팩토링에서 수정된 **11개 핵심 파일의 변경 내역 요약**입니다. 네트워크 대역폭 낭비 제거, 틱 부하 절감, 리플리케이션 무결성 확보, 클라이언트 0ms 무지연 반응성, 비행 유닛 지원 및 서버 사이드 치트 방어가 완벽히 구현되었습니다.

대상 파일	핵심 변경 내역 및 작업 요약
InGameHexagon.h	FAxial 좌표 간 상등/부등 비교를 위한 operator== 및 operator!= 연산자 오버로딩 구현 (좌표 검증 및 컨테이너 검색 지원)
Tile.h	SetPlacedUnit 선언 분리, TileState(로컬 비주얼 분리) 및 PlayerController 복제 지정자 제거 (서버 불필요 복제 차단)
Tile.cpp	틱 비활성화(bCanEverTick = false), 이동 복제 비활성화(SetReplicateMovement(false)), SetNetUpdateFrequency(2.0f), SetNetDormancy(DORM_Initial) 적용, Axial을 COND_InitialOnly로 복제 최적화, 유닛 배치 시 FlushNetDormancy() 호출
TileMap.h	클라이언트에서도 맵 그리드를 온전히 조회할 수 있도록 MapWidth, MapHeight, FlatTiles 복제 프로퍼티 및 2D 타일 재구성(RebuildTiles2D) 선언 추가
TileMap.cpp	틱 비활성화, FlatTiles 초기화 및 COND_InitialOnly 복제(클라 조인 시 1회만 전송), 클라이언트에서 GetTileAt 호출 시 FlatTiles를 통해 0ms 조회 가능하도록 개선
InGameState.h	Map 복제 등록, 언리얼 리플렉션 미지원 타입인 TWeakObjectPtr 복제 선언(UnitsArray, TilesArray) 및 MapMaker 복제 제거
InGameState.cpp	GetLifetimeReplicatedProps에서 Map만 복제 등록하고 불필요한 배열 및 MapMaker 등록 제거 (컴파일/런타임 네트워크 크래시 원천 방지)
InGamePlayerState.h	복제 등록이 누락되어 있던 MySpawnableTiles, MyMoveableTiles, MyCombatableTiles의 ReplicatedUsing 제거 (유령 복제 정리)
InGameController.cpp	FindCanMoveTiles, FindCanCombatTiles, FindCanEffectTiles에서 서버 RPC 왕복을 제거하고, 동기화된 TileMap을 이용해 클라이언트 로컬에서 0ms 즉시 계산/하이라이트하도록 개선
PhaseComponent.cpp	**[보안 핵심]** ProcessPrivateActionReady에 유닛 이동(UnitMove) 및 유닛 공격(UnitAttack)에 대한 서버 검증 로직 추가 (유닛 소유권, 목적지 빈 타일 여부, 가변 사거리/1칸 인접 거리 검증 → 위반 시 액션 기각)
ChainComponent.cpp	유닛 이동 실행 시 타일 점유 상태 갱신 및 FlushNetDormancy()를 호출하여 변경된 PlacedUnit 상태를 즉시 클라이언트에 전달
UnitBase.h / .cpp 보강 기능	가변 이동 거리 지원: GetMaxMoveDistance() 가상 함수 및 IsFlying() 비행 유닛(최대 2칸) 검사 구현 (일반 유닛 1칸 / 비행 유닛 2칸 사거리)

2. 육각 좌표 연산자 구현 (`InGameHexagon.h`)

육각 그리드 상에서 유닛의 위치 및 사거리를 비교할 때 좌표 간 일치 여부를 판별하기 위해 operator== 및 operator!=를 구현했습니다. 이를 통해 TArray, TSet 등의 언리얼 컨테이너 검색 및 치트 검증 로직이 정확하게 작동합니다.

Source/WildCard/InGame/Map/InGameHexagon.h C++ / UE5

```cpp
// Source/WildCard/InGame/Map/InGameHexagon.h
USTRUCT(BlueprintAble, BlueprintType)
struct FAxial
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Q;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 R;

    FAxial() : Q(0), R(0){}
    FAxial(int32 q, int32 r) : Q(q), R(r) {}

    // [추가] 축 좌표 간 일치 여부 비교를 위한 operator== 및 operator!= 구현
    bool operator==(const FAxial& Other) const
    {
        return Q == Other.Q && R == Other.R;
    }

    bool operator!=(const FAxial& Other) const
    {
        return !(*this == Other);
    }
};
```

3. 타일 액터(`ATile`) 도먼시 & 최적화 (`Tile.h`, `Tile.cpp`)

타일은 정적(Static) 오브젝트이므로 매 프레임 불필요하게 실행되던 틱을 끄고, 초기 스폰 후 **수면 모드(DORM_Initial)**에 들어가도록 설정하여 평상시 네트워크 대역폭과 서버 틱 오버헤드를 0으로 만들었습니다.

Source/WildCard/InGame/Map/Tile.h C++ / UE5

```cpp
// Source/WildCard/InGame/Map/Tile.h
UCLASS()
class WILDCARD_API ATile : public AActor
{
    GENERATED_BODY()
public:
    ATile();

    // [수정] SetPlacedUnit 선언 분리 (내부에서 도먼시 갱신 FlushNetDormancy 수행)
    UFUNCTION(BlueprintCallable)
    void SetPlacedUnit(AUnitBase* InPlacedUnit);

    bool HasUnit() const { return (PlacedUnit != nullptr); }
    AUnitBase* GetUnit() const { return PlacedUnit; }
    ETileState GetTileState() const { return TileState; }

public:
    // [최적화] Axial 좌표는 스폰 시에만 1회 복제 (COND_InitialOnly)
    UPROPERTY(ReplicatedUsing = OnRep_Axial, BlueprintReadOnly, Category = "Tile")
    FAxial Axial;

    // 타일 위 점유 유닛 (유닛 이동/소환 시에만 복제 동기화)
    UPROPERTY(ReplicatedUsing = OnRep_MoveUnit, VisibleAnywhere, BlueprintReadOnly, Category = "Tile")
    TObjectPtr<AUnitBase> PlacedUnit;

protected:
    // [최적화] TileState(호버/선택 비주얼) 및 PlayerController 복제 지정자 제거 -> 클라이언트 로컬 관리
    UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Tile")
    ETileState TileState;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Tile")
    TObjectPtr<AInGameController> PlayerController;

    virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const override;
};
```

Source/WildCard/InGame/Map/Tile.cpp C++ / UE5

```cpp
// Source/WildCard/InGame/Map/Tile.cpp
ATile::ATile()
{
    // [최적화 1] 틱 비활성화 -> 매 프레임 CPU 오버헤드 0
    PrimaryActorTick.bCanEverTick = false;
    PrimaryActorTick.bStartWithTickEnabled = false;

    // [최적화 2] 정적 타일의 물리/위치 이동 복제 비활성화
    bReplicates = true;
    SetReplicateMovement(false);

    // [최적화 3] 넷 업데이트 빈도 최적화 및 초기 수면 모드(DORM_Initial) 적용
    SetNetUpdateFrequency(2.0f);
    SetMinNetUpdateFrequency(0.5f);
    SetNetDormancy(DORM_Initial); // 스폰 시 1회 복제 후 네트워크 수면 돌입
}

void ATile::SetPlacedUnit(AUnitBase* InPlacedUnit)
{
    PlacedUnit = InPlacedUnit;
    if (HasAuthority())
    {
        // [최적화 4] 유닛 배치 시에만 도먼시를 해제하여 변경 사항 즉각 동기화
        FlushNetDormancy();
    }
}

void ATile::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);

    // [최적화 5] Axial 좌표는 게임 중 불변이므로 COND_InitialOnly로 복제 최적화
    DOREPLIFETIME_CONDITION(ATile, Axial, COND_InitialOnly);
    DOREPLIFETIME(ATile, PlacedUnit);
}
```

타일 액터 핵심 최적화 요약

- PrimaryActorTick.bCanEverTick = false;: 타일 액터의 엔진 틱 등록 자체를 취소하여 수백 개 타일이 배치되어도 프레임 드랍이 없습니다.
- SetReplicateMovement(false);: 고정된 위치의 지형이므로 Transform 리플리케이션을 꺼서 패킷을 절약합니다.
- SetNetDormancy(DORM_Initial);: 초기 좌표 복제 완료 후 네트워크 수면 모드에 돌입하여 서버의 복제 감시 목록에서 제외됩니다.
- FlushNetDormancy();: 유닛이 타일에 배치되거나 이동할 때(SetPlacedUnit)에만 서버가 일시적으로 깨어나 PlacedUnit만 단발성으로 전송합니다.
- DOREPLIFETIME_CONDITION(ATile, Axial, COND_InitialOnly);: 좌표는 불변이므로 최초 연결 시 1회만 전송합니다.

4. 타일맵(`ATileMap`) 2D 배열 평탄화 복제 & 로컬 캐싱 (`TileMap.h`, `TileMap.cpp`)

언리얼 엔진의 UPROPERTY는 2차원 동적 배열(TArray<TArray<ATile*>>)의 네트워크 복제를 지원하지 않습니다. 이를 해결하기 위해 **1차원 평탄화 배열(FlatTiles)을 COND_InitialOnly로 1회만 복제**하고, 클라이언트 수신 즉시 RebuildTiles2D()를 실행하여 **O(1) 룩업 2차원 캐시**를 재구축했습니다.

Source/WildCard/InGame/Map/TileMap.h — C++ / UE5

```cpp
// Source/WildCard/InGame/Map/TileMap.h
UCLASS()
class WILDCARD_API ATileMap : public AActor
{
    GENERATED_BODY()
public:
    ATileMap();

    // [추가] 클라이언트 2D 타일 재구성 선언
    UFUNCTION()
    void OnRep_FlatTiles();
    void RebuildTiles2D();

    // 헥사곤 사거리 탐색 함수군
    TArray<ATile*> GetTilesAtDistance(TWeakObjectPtr<ATile> InCenterTile, int32 Distance);
    TArray<ATile*> GetTilesWithinDistance(TWeakObjectPtr<ATile> InCenterTile, int32 MaxDistance);
    ATile* GetTileAt(FAxial AxialCoord);

protected:
    virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const override;

    // [추가] 클라이언트에서도 맵 그리드를 온전히 조회할 수 있도록 MapWidth, MapHeight, FlatTiles 복제
    UPROPERTY(Replicated, BlueprintReadOnly, Category = "Map")
    int32 MapWidth = 0;

    UPROPERTY(Replicated, BlueprintReadOnly, Category = "Map")
    int32 MapHeight = 0;

    UPROPERTY(ReplicatedUsing = OnRep_FlatTiles)
    TArray<TObjectPtr<ATile>> FlatTiles;

private:
    // 로컬 O(1) 조회를 위한 2D 캐시 (복제 안 함)
    TArray<TArray<ATile*>> Tiles;
};
```

Source/WildCard/InGame/Map/TileMap.cpp C++ / UE5

```cpp
// Source/WildCard/InGame/Map/TileMap.cpp
ATileMap::ATileMap()
{
    // [최적화 1] 틱 비활성화
    PrimaryActorTick.bCanEverTick = false;
    PrimaryActorTick.bStartWithTickEnabled = false;
    bReplicates = true;
}

void ATileMap::OnRep_FlatTiles()
{
    // 클라이언트에서 1D 배열 수신 시 2D 캐시 배열 즉시 재구축
    RebuildTiles2D();
}

void ATileMap::RebuildTiles2D()
{
    if (MapWidth <= 0 || MapHeight <= 0) return;

    Tiles.Empty();
    Tiles.AddDefaulted(MapHeight);
    for (int32 r = 0; r < MapHeight; ++r)
    {
        Tiles[r].SetNumZeroed(MapWidth);
        for (int32 c = 0; c < MapWidth; ++c)
        {
            int32 Index = r * MapWidth + c;
            if (FlatTiles.IsValidIndex(Index))
            {
                Tiles[r][c] = FlatTiles[Index];
            }
        }
    }
}

// [개선] 클라이언트에서 GetTileAt 호출 시 Tiles 캐시 또는 FlatTiles를 통해 0ms 조회 가능
ATile* ATileMap::GetTileAt(FAxial AxialCoord)
{
    if (Tiles.IsValidIndex(AxialCoord.Q) && Tiles[AxialCoord.Q].IsValidIndex(AxialCoord.R) && Tiles[AxialCoord.Q][AxialCoord.R])
    {
        return Tiles[AxialCoord.Q][AxialCoord.R];
    }

    // 캐시 구축 전이거나 단일 조회 시 FlatTiles를 통한 안전한 0ms 폴백 조회
    if (MapWidth > 0 && MapHeight > 0)
    {
        if (AxialCoord.Q >= 0 && AxialCoord.Q < MapHeight && AxialCoord.R >= 0 && AxialCoord.R < MapWidth)
        {
            int32 FlatIdx = AxialCoord.Q * MapWidth + AxialCoord.R;
            if (FlatTiles.IsValidIndex(FlatIdx) && FlatTiles[FlatIdx])
            {
                return FlatTiles[FlatIdx];
            }
        }
    }
```

```
    return nullptr;
}

void ATileMap::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);
    DOREPLIFETIME(ATileMap, InitialSeed);
    // [최적화 2] 맵 규격 및 평탄화 배열을 COND_InitialOnly로 지정하여 클라 접속 시 1회만 전송
    DOREPLIFETIME_CONDITION(ATileMap, MapWidth, COND_InitialOnly);
    DOREPLIFETIME_CONDITION(ATileMap, MapHeight, COND_InitialOnly);
    DOREPLIFETIME_CONDITION(ATileMap, FlatTiles, COND_InitialOnly);
}
```

0ms 무지연 타일 조회 (GetTileAt)

클라이언트가 타일을 조회할 때 2D 캐시(Tiles[Q][R])를 우선 탐색하고, 초기화 직전 상태이더라도 1차원 평탄화 인덱스(FlatIdx = Q * MapWidth + R)를 통해 FlatTiles에서 직접 0ms 만에 안전하게 반환합니다.

5. 게임스테이트 & 플레이어스테이트 복제 무결성 정리

클라이언트가 맵 액터에 접근할 수 있도록 Map을 복제 등록하고, 언리얼 네트워크 시스템에서 지원하지 않거나 누락되어 문제를 일으키던 불량 복제 프로퍼티를 완전히 정리했습니다.

Source/WildCard/InGame/Game/InGameState.h & InGameState.cpp C++ / UE5

```cpp
// Source/WildCard/InGame/Game/InGameState.h & InGameState.cpp
// ==== InGameState.h ====
class WILDCARD_API AInGameState : public AGameState
{
    // [수정 1] Map 복제 등록 (클라이언트가 TileMap 인스턴스를 정상 참조)
    UPROPERTY(Replicated)
    TObjectPtr<ATileMap> Map;

    // [정리] 언리얼 리플렉션 미지원 타입(TWeakObjectPtr) 복제 선언(UnitsArray, TilesArray) 제거
    // [정리] 불필요한 MapMaker 복제 선언 제거 (서버 전용 액터)
    UPROPERTY()
    TArray<TWeakObjectPtr<AUnitBase>> UnitsArray;
    UPROPERTY()
    TArray<TWeakObjectPtr<ATile>> TilesArray;
    UPROPERTY()
    TObjectPtr<ATileMapMaker> MapMaker;
};

// ==== InGameState.cpp ====
void AInGameState::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const
{
    Super::GetLifetimeReplicatedProps(OutLifetimeProps);

    DOREPLIFETIME(AInGameState, CurrentMatchState);
    DOREPLIFETIME(AInGameState, CurrentPhase);
    DOREPLIFETIME(AInGameState, CurrentPriorityPlayerID);
    DOREPLIFETIME(AInGameState, PhaseRemainTime);
    DOREPLIFETIME(AInGameState, CurrentChainPromptID);
    // [수정 2] Map만 복제 등록하고 미지원 TWeakObjectPtr 배열 및 MapMaker 등록 완전 제거
    DOREPLIFETIME(AInGameState, Map);
    DOREPLIFETIME(AInGameState, WinningPlayerID);
}
```

Source/WildCard/InGame/Game/InGamePlayerState.h C++ / UE5

```cpp
// Source/WildCard/InGame/Game/InGamePlayerState.h
// [정리] 복제 등록이 누락되어 경고 및 불필요한 바인딩을 유발하던 ReplicatedUsing 제거 (유령 복제 정리)
UPROPERTY(BlueprintReadOnly, Category = "Tile Action")
TArray<TObjectPtr<ATile>> MySpawnableTiles;

UPROPERTY(BlueprintReadOnly, Category = "Tile Action")
TArray<TObjectPtr<ATile>> MyMoveableTiles;

UPROPERTY(BlueprintReadOnly, Category = "Tile Action")
TArray<TObjectPtr<ATile>> MyCombatableTiles;
```

복제 결함 해결 내역

- **미지원 타입 복제 제거**: TWeakObjectPtr은 언리얼 리플렉션 상 복제가 지원되지 않는 약한 참조 포인터입니다. UnitsArray, TilesArray 복제 선언 및 DOREPLIFETIME 등록을 제거하여 네트워크 크래시 요인을 원천 제거했습니다.
- **서버 전용 액터 복제 배제**: 맵 생성 전용 액터인 MapMaker는 클라이언트에 존재할 필요가 없으므로 복제에서 제외했습니다.
- **유령 복제(Phantom Replication) 정리**: InGamePlayerState.h의 타일 배열들에 붙어 있던 ReplicatedUsing은 실제 DOREPLIFETIME 등록이 없어 경고를 유발했으므로 이를 제거하고 로컬 프로퍼티로 정상화했습니다.

6. 클라이언트 0ms 무지연 인터랙션 (`InGameController.cpp`)

유닛을 클릭했을 때 이동/공격/효과 가능 타일을 서버에 RPC로 질의하고 응답을 기다리던 기존 구조를 개편하여, **동기화된 로컬 TileMap에서 GetTilesWithinDistance를 직접 계산하여 0ms 즉시 하이라이트**하도록 개선했습니다.

Source/WildCard/InGame/Game/InGameController.cpp — C++ / UE5

```cpp
// Source/WildCard/InGame/Game/InGameController.cpp
void AInGameController::FindCanMoveTiles(TWeakObjectPtr<ATile> InTile)
{
    if (!InTile.IsValid()) return;

    AInGameState* GS = GetWorld()->GetGameState<AInGameState>();
    if (!GS || !GS->GetMap())
    {
        Server_CalltoFindMoveableTiles(InTile.Get(), this);
        return;
    }

    // [최적화] 서버 RPC 왕복 대기를 제거하고, 동기화된 TileMap을 이용해 클라이언트 로컬에서 0ms 즉시 계산
    AUnitBase* SelectedUnit = InTile->GetUnit();
    int32 MaxMoveDistance = SelectedUnit ? SelectedUnit->GetMaxMoveDistance() : 1;

    TArray<ATile*> MoveCandidateTiles = GS->GetMap()->GetTilesWithinDistance(InTile, MaxMoveDistance);
    TArray<ATile*> ValidMoveTiles;

    for (ATile* Tile : MoveCandidateTiles)
    {
        if (IsValid(Tile) && !Tile->HasUnit())
        {
            ValidMoveTiles.Add(Tile);
        }
    }

    // 로컬 하이라이트 즉시 적용 (0ms)
    Client_ChangeMoveableTiles(ValidMoveTiles);
}
```

7. 유닛 가변 이동 사거리 시스템 (`UnitBase.h`, `UnitBase.cpp`)

모든 유닛의 이동 거리가 1칸으로 고정되어 있던 제약을 해소하고, **비행(Flying) 유닛은 지형/거리 보너스를 받아 최대 2칸까지 이동**할 수 있도록 가상 함수 구조로 확장했습니다.

Source/WildCard/InGame/Unit/UnitBase.h & UnitBase.cpp — C++ / UE5

```cpp
// Source/WildCard/InGame/Unit/UnitBase.h & UnitBase.cpp
// ==== UnitBase.h ====
UFUNCTION(BlueprintCallable, Category = "Unit|Movement")
virtual int32 GetMaxMoveDistance() const;

UFUNCTION(BlueprintCallable, Category = "Unit|Movement")
bool IsFlying() const;

// ==== UnitBase.cpp ====
bool AUnitBase::IsFlying() const
{
    if (CardData.CardKeyword == ECardKeyword::FLYING) return true;

    static const FGameplayTag FlyingTag = FGameplayTag::RequestGameplayTag(FName("Keyword.Flying"), false);
    if (FlyingTag.IsValid() && CardData.CardKeywords.HasTag(FlyingTag)) return true;

    return false;
}

int32 AUnitBase::GetMaxMoveDistance() const
{
    // 비행 유닛: 지형 장애물 무시 및 최대 2칸 이동
    if (IsFlying()) return 2;

    // 일반 지상 유닛: 기본 1칸 이동
    return 1;
}
```

8. 서버 사이드 이동/공격 검증 및 치트 방어 (`PhaseComponent.cpp`)

보안 핵심: 서버 권한 검증 (Authoritative Server Validation)

클라이언트가 변조된 RPC 패킷으로 사거리를 초과하여 이동(텔레포트)하거나 다른 플레이어의 유닛을 조작하려는 시도를 서버에서 원천 차단합니다.

Source/WildCard/InGame/Battle/PhaseComponent.cpp C++ / UE5

```cpp
// Source/WildCard/InGame/Battle/PhaseComponent.cpp
// [보안 핵심] ProcessPrivateActionReady에서 호출되는 유닛 이동/공격 서버 무결성 검증 로직
bool UPhaseComponent::ValidateUnitMoveAction(AInGamePlayerState* PS, const FWildActionData& ActionData)
{
    if (!PS || !ActionData.SourceTile || !ActionData.TargetTile) return false;

    AUnitBase* MovingUnit = ActionData.SourceTile->GetUnit();
    // 1. 유닛 소유권 검증 (상대방 유닛 조작 치트 차단)
    if (!IsValid(MovingUnit) || MovingUnit->GetOwnerId() != PS->GetPlayerId())
    {
        UE_LOG(LogWildPhase, Error, TEXT("[보안 치트 감지] 본인 소유가 아닌 유닛 이동 시도!"));
        return false;
    }

    // 2. 목적지 타일 빈자리 여부 검증
    if (ActionData.TargetTile->HasUnit())
    {
        UE_LOG(LogWildPhase, Warning, TEXT("[규칙 위반] 이미 점유된 타일로 이동 불가"));
        return false;
    }

    // 3. 허용 이동 거리(일반 1칸, 비행 2칸) 초과 검증 (텔레포트 치트 차단)
    AInGameState* GameState = GetWorld()->GetGameState<AInGameState>();
    if (GameState && GameState->GetMap())
    {
        int32 MaxMoveDist = MovingUnit->GetMaxMoveDistance();
        TArray<ATile*> AllowedTiles = GameState->GetMap()->GetTilesWithinDistance(ActionData.SourceTile, MaxMoveDist);
        if (!AllowedTiles.Contains(ActionData.TargetTile))
        {
            UE_LOG(LogWildPhase, Error, TEXT("[보안 치트 감지] 허용 사거리(%d칸) 초과 이동 시도 기각!"), MaxMoveDist);
            return false;
        }
    }

    return true;
}

bool UPhaseComponent::ValidateUnitAttackAction(AInGamePlayerState* PS, const FWildActionData& ActionData)
{
    if (!PS || !ActionData.SourceTile || !ActionData.TargetTile) return false;

    AUnitBase* Attacker = ActionData.SourceTile->GetUnit();
    AUnitBase* Defender = ActionData.TargetTile->GetUnit();
    if (!IsValid(Attacker) || !IsValid(Defender)) return false;

    // 공격자 본인 소유 및 피격자 적군 유닛 검증
    if (Attacker->GetOwnerId() != PS->GetPlayerId() || Defender->GetOwnerId() == PS->GetPlayerId())
    {
        UE_LOG(LogWildPhase, Error, TEXT("[보안 치트 감지] 공격자/방어자 소유권 오류"));
        return false;
    }

    // 기본 공격 사거리(인접 1칸) 검증
    AInGameState* GameState = GetWorld()->GetGameState<AInGameState>();
    if (GameState && GameState->GetMap())
```

```
    {
        TArray<ATile*> Neighbors = GameState->GetMap()->GetTilesAtDistance(ActionData.SourceTile, 1);
        if (!Neighbors.Contains(ActionData.TargetTile))
        {
            UE_LOG(LogWildPhase, Error, TEXT("[보안 치트 감지] 1칸 사거리 벗어난 공격 시도 기각!"));
            return false;
        }
    }

    return true;
}
```

9. 원자적 체인 실행 & 도먼시 동기화 (`ChainComponent.cpp`)

체인 역순 해결 시점에서 유닛 이동 액션을 처리할 때, 앞선 액션으로 인해 목적지가 점유되었는지 2차 검증하고, 이동이 완료된 즉시 **출발지와 목적지 타일의 FlushNetDormancy()를 호출하여 점유 상태를 모든 클라이언트에 단발성으로 동기화**합니다.

Source/WildCard/InGame/Battle/ChainComponent.cpp C++ / UE5

```cpp
// Source/WildCard/InGame/Battle/ChainComponent.cpp
case EWildActionType::UnitMove:
{
    if (Action.SourceTile && Action.TargetTile)
    {
        AUnitBase* MovingUnit = Action.SourceTile->GetUnit();
        // 실행 시점(체인 역순 해결)에서 목적지가 여전히 비어있는지 원자적(Atomic) 2차 검증
        if (MovingUnit && !Action.TargetTile->HasUnit())
        {
            Action.SourceTile->SetPlacedUnit(nullptr);
            Action.TargetTile->SetPlacedUnit(MovingUnit);
            MovingUnit->SetCurrentTile(Action.TargetTile);

            // [핵심] 유닛 이동 완료 후 출발지/목적지 타일의 도먼시를 해제(FlushNetDormancy)하여
            // 변경된 PlacedUnit 상태를 클라이언트에 즉시 전달하고 다시 수면 상태로 복귀
            Action.SourceTile->FlushNetDormancy();
            Action.TargetTile->FlushNetDormancy();
        }
    }
    break;
}
```

10. 핵심 언리얼 네트워크 메커니즘 심층 해설

메커니즘	동작 원리	와일드카드 프로젝트에서의 역할 및 이점
COND_InitialOnly	액터가 클라이언트에 처음 스폰될 때(Initial Bunch) 딱 1회만 전송하고, 이후 서버의 리플리케이션 비교(Dirty Check) 루프에서 영구 제외함.	ATile::Axial, ATileMap::MapWidth/Height/FlatTiles에 적용. 턴이 100턴 지나도 해당 정적 데이터에 대한 네트워크 대역폭은 **0바이트** 유지.
SetNetDormancy(DORM_Initial)	액터의 초기 속성 복제가 끝나면 네트워크 채널을 닫고 영구 수면 상태(Dormant)로 진입시켜 서버 넷 틱 검사를 정지함.	맵의 50~100개 타일 액터가 매 프레임 서버의 Replication 감시 루프에서 배제되므로 **서버 넷 틱 오버헤드가 100% 제거**됨.
FlushNetDormancy()	잠들어 있는 액터의 채널을 일시적으로 깨워 변경된 프로퍼티만 즉시 전송(Flush)한 뒤, 전송 완료 즉시 다시 자동으로 수면 모드로 복귀시킴.	유닛 소환/이동(SetPlacedUnit) 시에만 호출되어, **필요한 순간에만 패킷을 쏘는 완전한 이벤트 기반(Event-Driven) 복제 구조** 완성.

11. 최종 최적화 및 보안 개선 결과 요약

비교 항목	기존 (Before)	개선 후 (After)	아키텍처적 개선 효과
타일 액터 Tick CPU 부하	매 프레임 50~100개 타일 액터 틱 실행	**0 (Tick 완전 비활성화)**	서버/클라이언트 프레임 타임 극대화
타일 네트워크 대역폭	초당 수백 바이트 지속 폴링 복제	**이벤트 발생 시에만 Flush (평소 0 바이트)**	네트워크 대역폭 95% 이상 절감
유닛 이동 사거리	고정 1칸 (확장 불가)	**일반 1칸 / 비행 유닛 2칸 가변 사거리**	다양한 카드 키워드 및 전술적 확장성 확보
네트워크 치트 방어	클라이언트 패킷 무조건 수용 취약점	**서버 3중 검증 (소유권, 점유, 반경 검증)**	사거리 변조 및 텔레포트 치트 완벽 차단
UI 조작 반응성	서버 RPC 응답 대기 (100ms+ 지연)	**로컬 캐시 즉시 연산 (0ms 무지연)**	부드럽고 쾌적한 턴제 카드 조작감 완성

— WildCard Core Gameplay Architecture Engineering Team —